

Traceable Server States with Git and Nix

Bachelor of Engineering Thesis



Traceable Server States with Git and Nix

Bachelor of Engineering Thesis
December, 2025

By
Henrik Erik Albert Zenkert

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.

Cover photo: Vibeke Hempler, 2012

Published by: DTU, Department of Applied Mathematics and Computer Science,
Anker Engelunds Vej 101, 2800 Kgs. Lyngby Denmark
www.byg.dtu.dk

ISSN: [0000-0000] (electronic version)

ISBN: [000-00-0000-000-0] (electronic version)

ISSN: [0000-0000] (printed version)

ISBN: [000-00-0000-000-0] (printed version)

Approval

This thesis was prepared over four months at DTU Compute, Technical University of Denmark (DTU), in collaboration with DBC Digital, as part of the requirements for the Bachelor of Engineering in Software Technology.

It is assumed that the reader has a basic understanding of distributed systems, software architecture, and the Nix/NixOS ecosystem.

Henrik Erik Albert Zenkert - s224281

.....
Signature

.....
Date

Abstract

Modern distributed infrastructures increasingly require reproducibility and traceability to ensure reliable and predictable operation across large server fleets. However, existing tools often fall short because they cannot provide a complete auditable overview of what has been running over time. This thesis addresses that challenge in the context of NixOS-based systems deployed across multiple data centres.

The project designs and implements a system for tracking which built NixOS system is active on each host over time, and linking each deployed system's unique identifier to the Git revision that produced it.

The system is designed to solve the following primary tasks:

1. verifying whether all hosts in a cluster are running systems built from the same Git revision
2. identifying which hosts change, and which remain unchanged, when a centrally managed system configuration shared across many hosts is updated

The system, called *hostmap*, uses a decoupled architecture in which a central service collects unique identifiers for the currently running system from NixOS hosts, and receives mapping data from an external source (for example, a CI pipeline) that links these identifiers to Git revisions.

This results in a practical system for tracking runtime deployment provenance in NixOS environments, which improves visibility into the running infrastructure and simplifies auditing and troubleshooting in large-scale NixOS server fleets.

Acknowledgements

A big thank you to everyone who supported me in this project:

Christian Gram Kalhauge, Assistant Professor, DTU

Without your help, guidance, and support, there would be no thesis or project. Your help in structuring this thesis and keeping the focus of the project on the right things really made a world of difference.

Adam Tulinius, Infrastructure Developer, DBC Digital

Your mentoring, patience, and competence were invaluable in making this project a success.

Sarah Brofeldt, Chief Infrastructure Developer, DBC Digital

Thank you for helping me scratch the surface of Nix.

The Infrastructure Team at DBC Digital

Seeing what the team accomplishes with Nix and other technologies makes me realise how much there is to learn.

I would also like to thank DBC Digital for being welcoming and supportive. Having the chance to work with Nix and Rust to deploy code I contributed to more than 200 servers in production has been a rare and genuinely exciting experience. Knowing that almost the entire Danish library infrastructure now runs code I contributed has been both exciting and deeply humbling.

Contents

Preface	ii
Abstract	iii
Acknowledgements	iv
1 Introduction	2
1.1 Contributions	3
2 Problem Description and System Requirements	5
2.1 System Requirements	7
2.1.1 Functional Requirements	7
2.1.2 Non-Functional Requirements	7
2.2 Nix Package Manager and NixOS	8
2.3 Why We Need Mappings	9
2.4 Related Technologies	10
2.4.1 Previous System — hostoverview	10
2.4.2 Morph	10
2.4.3 OpenTelemetry	11
2.4.4 Prometheus	11
2.4.5 Provenance and Deployment Tracking Systems	12
3 Methodology	13
3.1 Overview	13
3.2 Iterative Development Model	13
3.2.1 Requirement Clarification	14
3.2.2 Design and Planning	14
3.2.3 Implementing a Minimal Increment	14
3.2.4 Validation and Feedback	15
3.2.5 Finding the Next Step	15
3.3 Advantages of the Iterative Methodology	15
3.4 Application in this Project	16
4 Process	17
4.1 Overview	17
4.2 Early Development — Mid September	17
4.3 Completing Activation Log Pipeline and First UI — Late September	18
4.4 Server and UI Development — Early October	18
4.5 Adding Metadata — Early to Mid October	19
4.6 Usability Improvements — Mid October	19

4.6.1	Timing-Attack Mitigation	20
4.7	CI Server Integration — Mid October to Late November	20
4.8	Project Slowdown — Late October to Mid November	20
4.9	Final Code — Mid November	21
4.10	Deployment to Test — End of November	21
4.11	Deployment to Production — Start of December	21
4.12	Conclusion	22
5	Solution	24
5.1	Data Flow	24
5.1.1	Pipeline Dependencies	25
5.2	Architecture and Technology Choices	26
5.2.1	Architecture	26
5.2.2	Technology Choices	26
5.3	Activation Logger	28
5.4	Scraper	28
5.5	Server	29
5.5.1	API Design	29
5.5.2	Database	30
5.5.3	Web Interface	30
5.6	Nix Integration	30
5.7	Hostmap Deployed at DBC Digital	31
5.8	Summary	31
6	Evaluation	33
6.1	Testing and Validation	33
6.1.1	Unit Tests	33
6.1.2	Local Integration Tests	34
6.1.3	Mapping Endpoints Simulation	35
6.1.4	Integrating CI-Server and Load Testing	35
6.2	Manual End-to-End Testing	35
6.3	Security	36
6.4	Data Freshness	36
6.5	User Evaluation	36
6.5.1	Feedback from Infrastructure Developers	36
6.5.2	Summary	37
7	Discussion	38
7.1	Why Hostmap Fills a Gap	38
7.2	Strengths of Hostmap	39
7.3	Trade-offs and Limitations	40
7.4	Limitations of Using Telemetry for Runtime Provenance	40
7.5	Future Work	41
7.6	Conclusion	42
	Bibliography	43
A	SQL Query	45

B	Feedback in Danish	46
B.1	Sarah Brofeldt — Lead Infrastructure Developer	46
B.2	Jonathan Zielinski — Infrastructure Developer	46
B.3	Erik Skytthe — Infrastructure Developer	46

1 Introduction

DBC Digital operates a fleet of NixOS-based servers whose system configurations are specified, built, and deployed from a central Git repository. In such an environment, reproducibility and traceability are essential. Reproducibility ensures that identical configurations create identical system images. Traceability is useful to understand how the running infrastructure changes over time.

To achieve traceability, infrastructure engineers must be able to determine exactly which system image is currently running on each server, and maintain a history of active system images over time. To make this information useful in practice, each system image identifier (Nix store path) must be traceable to the Git revision from which it was built. This requires runtime provenance, which records the system activations (changing of system image), combined with build-time provenance which links Nix store paths to Git revisions with reproducibility to ensure the mapping is correct.

So how can runtime provenance be provided for a fleet of NixOS hosts?

Currently, this runtime provenance is created through a fragmented process. The CI server builds the system image in a CI/CD (Continuous Integration/Continuous Delivery) pipeline for each commit. It then appends the mapping between the system image's Nix store path and the Git revision it originated from to a CSV file.

Meanwhile, activation data is generated by the hosts themselves. Whenever a system activation is executed on a host, a script runs that logs the activation data to a CSV file. The activation data is then gathered by a scheduled job on the CI server. The CI job accesses each host sequentially to retrieve the CSV file with system activations, and appends the new information to a file on the CI server. The two datasets are then correlated by a PHP-based tool to generate an overview of the hosts.

While this approach works in day-to-day operations, it has several limitations. It requires broad SSH access across the infrastructure, relies on CSV as a primary data store instead of a database, which limits querying and historical capabilities, and is difficult to extend or integrate with other systems. Due to the sequential and slow nature of the SSH scraping, the freshness of the data is poor, with up to 25-minute delays before a newly activated system shows up in the overview. As DBC Digital's needs evolve, these issues have become evident.

To address this gap, I designed and implemented an improved system providing an overview of hosts, called *hostmap*. It stores mappings between Nix store paths and the Git revisions that produced them, as well as runtime activation data from all hosts in the fleet, in a central service. It then correlates the activation data with the mapping data to provide a web interface for viewing the current and historical state

of the monitored hosts.

This enables two key capabilities: verifying that all hosts in a cluster are running systems built from the same Git revision, and observing which hosts change, and which remain unchanged, when a system configuration shared across many hosts is updated.

I designed hostmap to be suitable for open-source release. This is in line with DBC Digital's open-source culture and has influenced architectural decisions, e.g., avoiding hard-coding any organisation-specific things.

I will evaluate the new system by comparing its reliability, security, and usability with the existing SSH-based setup. A successful solution will match the existing tooling's functionality while significantly improving security, data freshness, and long-term operability.

1.1 Contributions

I, Henrik Zenkert, did this project individually, and I am the only one responsible for all parts of the thesis. This section shows my work as a software engineer reflected in the chapters and summarises my core contributions, which were:

- I analysed the legacy SSH and PHP-tool DBC Digital used and identified its limitations. The limitations concerned security and data freshness (Chapter 2).
- I introduced and explained the necessary background knowledge about Nix and NixOS, so that the reader can understand the problem hostmap solves (Chapter 2).
- I defined the functional and non-functional requirements for the new system. These included runtime provenance, persisting the data the runtime provenance depends on, and detection of identical systems across different commits (Chapter 2).
- I selected, justified, and adapted an iterative development methodology to fit a company collaboration where the requirements and constraints evolve (Chapter 3).
- I documented and analysed the actual development process. This included analysing deviations from the ideal methodology and explaining how design decisions were shaped by user feedback (Chapter 4).
- I implemented hostmap as three components (Activation Logger, Scraper, and Server). All three were packaged and deployed using Nix to ensure reproducibility and a NixOS module to facilitate configuration and usage (Chapter 5).
- I implemented a web interface that supports showing the current state for all hosts and a historical view per host. I also added a search function and a workflow using nix-diff (Chapter 5).
- I designed and implemented a decoupled architecture to enable independent deployment of the Server and Scraper, where hosts expose activation data

over an HTTP endpoint, which the Scraper collects, and the Server joins with externally provided, but locally persisted nix-store-path-to-commit mappings in a relational database (Chapter 5).

- I evaluated performance, correctness, and robustness using unit tests, local integration tests, CI integration tests, performance tests, and user evaluation from the infrastructure team at DBC Digital (Chapter 6).
- I discussed the strengths, limits, and trade-offs of the system. I also examined why hostmap fills a gap and future work. The discussion also included alternative solutions such as open telemetry and how hostmap relates to other technologies such as SLSA (Chapter 7).

Forward references

The following list helps the reader navigate the thesis by summarising the chapters in which each contribution is described.

Chapter 2	Describes the current system setup at DBC Digital, the legacy workflow, and the system requirements that hostmap must satisfy.
Chapter 3	Describes the ideal methodology for this project and how the workflow would be if everything went according to plan.
Chapter 4	Describes the actual iterative development process used in practice, including key changes driven by feedback and constraints.
Chapter 5	Presents the final architecture and implementation, including the three components, the data flows, and the main technology choices.
Chapter 6	Evaluates the system through tests and deployment experience, and compares it with the legacy SSH-based setup.
Chapter 7	Discusses why hostmap fills a gap, the strengths and trade-offs, alternative approaches such as Open Telemetry, and future work.

2 Problem Description and System Requirements

This chapter describes the problem that hostmap addresses and turns the problem into actual requirements for the system. I first document DBC Digital’s current system for runtime provenance and its limitations, because the limitations imply which requirements the new system must fulfil. After that, I give an introduction to Nix and NixOS concepts that the thesis builds on, so I can focus on actual design and implementation in later chapters, instead of terminology. Finally, I present related technologies that are either used in the project or deliberately not used. The reasoning behind some of these decisions will be explained in later chapters.

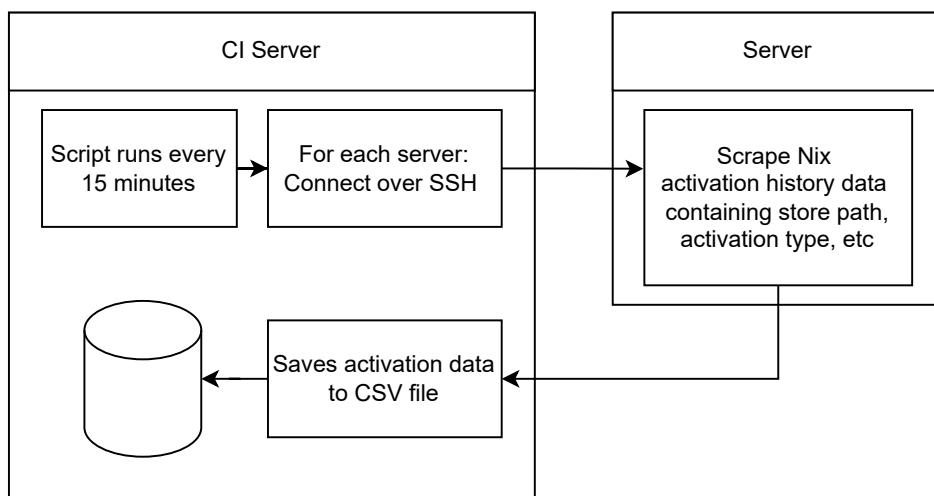


Figure 2.1: Scraping activation data over SSH in the legacy system

The legacy system uses a GitLab CI job that runs every 15 minutes. It collects logs with activation data over SSH from all hosts, as shown in Figure 2.1. This design leads to several consequences.

The first consequence is increased security risk. Because the data is collected over SSH, the CI server has SSH-access to all hosts, including production. This is a security concern. If the CI server is compromised, an attacker is given (unprivileged) SSH-access to all hosts.

A second consequence is reduced data freshness. The CI job runs every 15 minutes and takes approximately 10 minutes to complete. Thus, newly activated systems may take up to 25 minutes to be reflected in the data collected.

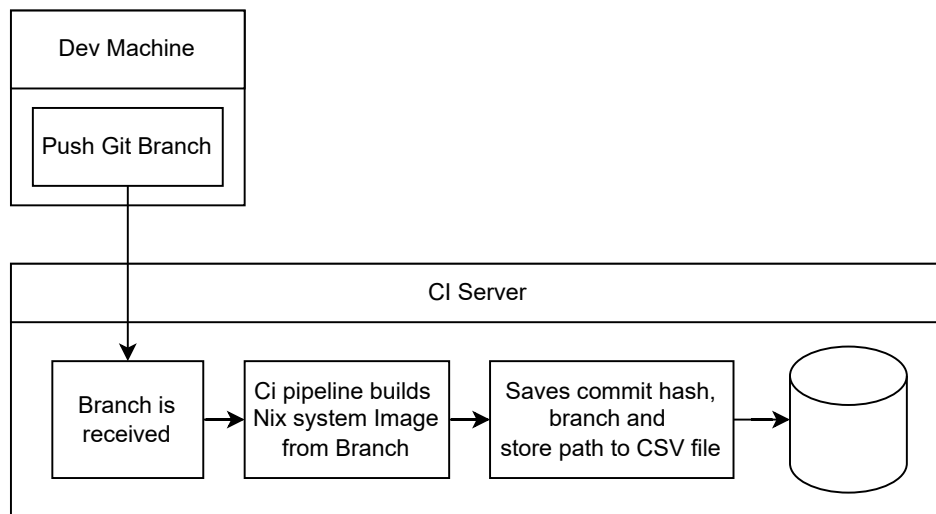


Figure 2.2: Creating mappings between Git commits and Nix store paths in the legacy system

This is problematic for the infrastructure team at DBC Digital because the tool is used to verify that hosts in a cluster are running the same version (same Git commit) after a deployment. During incident responses, the team use the overview to confirm that the hosts are running the expected Git revision. When there are up to 25-minute delays, distinguishing between a host that has not been updated and one that has updated but is not yet visible in the data can be hard.

A third consequence of the legacy system is that it cannot be deployed redundantly, something DBC Digital aims to do for all its systems. The legacy system relies on the CI server to not only build system images to generate mappings, but also to fetch activation data, and serve the overview of hosts. By concentrating these three responsibilities in a single component, the system introduces a single point of failure. If the CI server becomes unavailable, mapping generation, collecting activation data and serving the overview of hosts all become unavailable at the same time. This tight coupling also prevents independent scaling and deployment, meaning the scraping of activation data and the overview of hosts cannot be deployed in multiple instances without also replicating the CI server.

Together, these problems show that the existing solution does not give a reliable overview of hosts, and serves as a motivation for a new system that can securely collect activation data, mappings, and provide an overview of hosts.

Creating mappings between commit hash and store path is out of scope for the project, but how DBC Digital does it can be seen in Figure 2.2. My system will receive mappings and be responsible for linking them with the correct host and activation data. DBC Digital has decided that its CI Server will create the mapping, since it is already building the system image for other purposes.

2.1 System Requirements

In this project, *runtime provenance* is the ability to identify which software artifact a host is running at a given point in time and to associate the artifact with the source code revision it originates from. This association depends on *build-time provenance*, which provides verifiable guarantees about the origin of an artifact.

The system requirements are divided into functional and non-functional requirements.

2.1.1 Functional Requirements

The primary functional goal of the project is to provide runtime provenance for all hosts registered in hostmap. It does this through the following functional requirements:

- The system should support deployment of multiple instances of a component consisting of a server and database to improve availability, but not require exact consistency between them
- The system must collect and store activation events necessary to establish runtime provenance for all hosts
- The system must store externally generated build-time provenance mappings between Nix store paths and the Git revision and branch
- The system must detect when two different Git commits yield a functionally identical system image
- The system must allow end users to view the current Nix store path and revision for all hosts in a tabular overview
- The system must allow end users to view the historical relationship between Git commits and Nix store paths for each host
- The system must allow hosts to be grouped by a configurable key in the user interface

2.1.2 Non-Functional Requirements

- Security, the system must avoid the need for privileged access (as used in the current SSH-based solution) and must authenticate endpoint access where appropriate
- Reliability, must survive network interruptions
- Performance, must work well when deployed to 200 servers, each activating at most once per day on average
- Maintainability, simple deployment

2.2 Nix Package Manager and NixOS

To understand the problem, we first need to understand what Nix and NixOS are. NixOS is a Linux distribution built with the Nix package manager, which enables declarative and reproducible systems [1]. In Nix, the inputs required to build a package, e.g., source files, compilers, environment variables, libraries, and more, are specified in a `.nix` file. When using flakes (a newer workflow for Nix), all inputs are pinned to exact, immutable revisions in a `flake.lock` file [2].

To understand Nix, we can compare it to how pure functions in functional programming work. A pure function has no hidden state and, for the same input, will return the same output.

Drawing inspiration from this, Nix does the same for software builds. In your Nix expression, you list all of your required inputs, and for the same build inputs, the build artifact will, regardless of where, how, or when it is built, always be the same.

The build inputs all appear in the final store path of the artifact under `/nix/store/<hash>-<name>`, and the unique hash is calculated from all the build inputs.

A *Nix store path* refers to the unique path under `/nix/store` that identifies a build artifact and encodes a hash of all its build inputs.

If you change something small, such as an environment variable, the inputs will be altered, and thus the hash of the build artifact will be changed. This ensures reproducible and deterministic builds.

A build is *reproducible* if the same inputs produce identical outputs when built on different machines, and *deterministic* if a fixed set of inputs always yields the same output [3].

To ensure that the inputs themselves (which are also Nix expressions) have not been altered, Nix verifies all fetched sources against their expected hashes. If the computed hash of a source does not match the expected hash specified in the input, something unexpected has happened. The input might have been corrupted or tampered with, and the build fails.

```
1 flask==1.0.0
2 numpy==1.0.4
3 pillow==1.2.0
4 requests==1.3.1
5 cryptography==2.0.0
```

Figure 2.3: An example `requirements.txt` file that looks reproducible, but is not.

Contrast this with the approach of traditional package managers. There are many, e.g. `npm`, `pip`, `Maven`, and they all share the same problem. When you specify dependencies in a file, e.g., a `requirements.txt`, as you can see in Listing 2.3, the precise versions of packages are specified. The requirements file looks reproducible at first glance, since the exact versions are specified. There are, however, several problems. Two of them are: we do not know which version of Python we need, and libraries, such as `cryptography`, and `requests` depend on native system libraries, in this case, `OpenSSL`, and the version of `OpenSSL` is not specified [4].

These two problems may cause the classic “it works on my machine” problem and subtle bugs that are hard to diagnose.

By treating builds like pure functions and embedding all inputs directly into the Nix store paths, Nix enables strong reproducibility guarantees. These ideas form one of the foundations for the system developed in this project, which relies on the deterministic nature of Nix builds to support reliable provenance tracking and auditing.

You can use Nix as a package manager for individual projects or for installing applications on Linux systems. NixOS builds on these ideas by letting you describe the entire operating system in a declarative and reproducible way. Instead of creating users, installing programs, configuring groups, and enabling system services imperatively through shell commands, all the configuration and system state are declared in configuration file(s). From the configuration, Nix builds a complete system image in the Nix store. A *system image* is the complete, built NixOS system configuration stored in the Nix store, including the kernel and all system services and dependencies.

After building the system image, you can activate it. A (*system*) *activation* is the process of switching the running system to reference a specific Nix store path. All system activations in NixOS are performed through one entry point. They are all activated by running a script called `switch-to-configuration`. The script is invoked whenever a new system image is activated and is responsible for changing from one running system to the selected configuration.

All in all, this means that a system running NixOS can be uniquely identified by the Nix store path. If two configurations produce the same store paths, they are bit-by-bit identical in functionality, which forms the basis for `hostmap`.

2.3 Why We Need Mappings

A seemingly intuitive idea is to embed the current Git commit hash directly into the Nix configuration for a server. This seems to provide a simple and immediate way to map between a Git commit hash and its resulting store path.

However, this approach fails. At closer inspection, one sees that embedding the commit hash would cause the store path for a host in our fleet to change for *every* new commit, even when the commit does not introduce any functional change to the Nix configuration. The only difference is a small piece of metadata, the Git commit hash, but Nix would still treat this as a build input and produce a different store path [1].

This behaviour undermines one of the requirements for DBC Digital, namely the ability to detect when two different Git commits yield the same system image. If the store paths for all servers are always different for every commit solely because the commit hash changes, it would be impossible to see if a system is functionally unchanged, which is one of the key functional requirements for the system.

To summarise, embedding the commit hash pollutes the resulting store path value with irrelevant metadata. This makes every commit produce a unique store path, even when the resulting system is functionally the same. This causes the mapping between store paths and Git commits to be useless, and creates a need for a system outside the Nix configuration to record and relationally map this information.

Such a system can track which Git commit caused which store path and later map this data to activation events in the server fleet.

2.4 Related Technologies

2.4.1 Previous System — *hostoverview*

DBC used an internal system called *hostoverview* to present an overview of the NixOS hosts. The *hostoverview* tool ran on the CI Server and consisted of a PHP-based tool that read from the following files on the CI Server:

- a CSV file produced by the CI pipeline, creating the mappings between store path and Git commit revision
- a facts file with info about the hosts
- activation logs scraped from hosts via SSH and stored as flat files by a scheduled job on the CI Server
- a file containing the host group definitions

Using these inputs *hostoverview* generates a web interface displaying an overview of the hosts, together with historical activation information and commit mappings.

Over time, the CSV file with mappings had grown to several GB in size, and the freshness of the data in the web interface was quite poor due to the nature of the scheduled CI job collecting the data. On top of this, the old system required SSH access to hundreds of servers in production to collect the activation logs. This introduced a risk. If the CI server were compromised, the attacker could gain access (not root, but still a risk) to every host in the fleet. These things together motivated DBC Digital to find a new, better system.

2.4.2 Morph

Morph is an open-source deployment tool that DBC Digital has developed for NixOS hosts. It is a wrapper around Nix and SSH [5]. It enables automating building, copying, and activating Nix system images across a server fleet. Morph is convenient, since it enables deploying multiple hosts and clusters and filtering on which to deploy. For these reasons, Morph is a great tool when managing a fleet of NixOS hosts.

Morph deploys updates to hosts sequentially and then runs health checks on the host to ensure that the host is up before continuing to deploy the next one. This is useful when deploying an HA (High-Availability) cluster, to ensure not all machines are taken down simultaneously. In conjunction with other HA technologies, e.g. load balancers, Morph's sequenced deployments and health checks can be used as part of a workflow for zero-downtime deployments. Morph helps with deployment orchestration, but it does not show which system image is currently active on each host over time.

2.4.3 OpenTelemetry

OpenTelemetry (OTel) is an open-source framework for collecting and exporting telemetry signals such as metrics, logs, and traces. It provides a standardised protocol along with APIs and data formats for sending telemetry from infrastructure components and applications [6]. In addition, OpenTelemetry includes a stand-alone component called the OpenTelemetry Collector, which receives, processes, and exports this data. On top of this, the Collector acts as a pipeline that can batch, filter, process, and then forward telemetry data to analysis back ends.

OpenTelemetry defines three types of primary telemetry signals: metrics, logs, and traces.

Metrics are aggregated measurements collected over time, used for monitoring system health and trends, not to capture discrete events.

Traces track the execution path of operations, such as a request, over time. It consists of spans, and the spans represent an operation with a start and end timestamp.

Logs are discrete records of events with timestamps within a system. In logs, you can see individual events rather than aggregated values or how a request is executed in a system over time.

Telemetry data is typically transported asynchronously using a push-based model.

The OTLP (OpenTelemetry Protocol) 1.9.0 specification explicitly limits guarantees of data delivery to a single client-to-server hop:

“End-to-end delivery guarantees in such systems is outside of the scope of OTLP. The acknowledgements described in this protocol happen between a single client/server pair and do not span intermediary nodes in multi-hop delivery paths.” [6]

This means, in multi-step OTel pipelines, e.g., `app → OTel agent → OTel Collector → back-end database`, there is no guarantee that the data you are sending will be delivered to the back-end database without data loss, only a best-effort attempt, which includes retries, will be made.

How this affects whether or not OpenTelemetry is suitable for runtime provenance is discussed further in Chapter 7

2.4.4 Prometheus

Another solution for monitoring systems and infrastructure is Prometheus. In contrast to OTel, Prometheus uses a pull-based model. In Prometheus, a central server scrapes HTTP endpoints exposed by monitored targets at regular intervals.

The endpoints expose their data as values, identified by a name and a set of labels [7]. While the endpoint can expose multiple samples, each sample is an individual value, meaning the endpoint typically cannot expose a list of structured values. This means that each scrape by the central server becomes a snapshot of the monitored host's state at a certain timestamp, not an authoritative record of state.

Since the data is collected periodically, a state change that occurs between scrapes might not be observed, and Prometheus does not have a built-in way of providing a lossless record of all states.

Prometheus is used widely for monitoring infrastructure health and for triggering alerts when an anomaly is detected, including at DBC Digital.

2.4.5 Provenance and Deployment Tracking Systems

There exist many different tools for tracking provenance. One such tool is SLSA (Supply-chain Levels for Software Artifacts), and it records and verifies how software artifacts are built. SLSA was created to address a similar problem to the problem Nix store paths solve. It records and verifies build-time provenance by use of cryptographically signed metadata [8].

Unlike Nix, SLSA does not enforce reproducibility. With SLSA, the verification is performed by the user. The first time a new artifact should be created on a trusted machine, where you can trust the cryptographic hash, and on subsequent builds, the artifact is compared to the trusted hash. This allows users of this artifact to trust it. While SLSA provides guarantees about provenance at build-time, similar to the build-time provenance built into Nix, it does not help us see when and how artifacts are changed in a system. SLSA does not provide a way to track which system configuration state a host is running.

GitOps is a framework for using code as infrastructure [9]. Server configurations are written as code in your version control software, and then CI/CD (Continuous Integration/Continuous Deployment) pipelines validate and then enact updates to the environment. This ensures that a system converges toward a known configuration, and any configuration drift is eventually corrected. Important to note is that we have a Git commit that says what we would like the system to run, but GitOps provides no way to see what build artifact is running on a server at a given time.

3 Methodology

This chapter describes the methodology that guided the project and explains why an iterative approach was chosen over a waterfall model. This is included to make explicit how I aimed to do the development, and to stand in contrast to the actual process described in Chapter 4.

The methodology for this project is based on an iterative development model. Rather than designing the full system at the outset and implementing it in a single pass, the project was structured as a series of short, repeatable cycles in which requirements were refined, prototypes were produced, and feedback came continuously. This approach ensures that the needs of the client and the work of the engineer never drift far apart, since the short iteration intervals keep the feedback loop tight, an advantage typical for iterative and agile development models [10].

3.1 Overview

The iterative model is a great fit for infrastructure-related projects, where the requirements evolve as new technical constraints are discovered. Working in this agile way means that instead of attempting to pre-empt all the problems and anticipate all the needs, we let the system evolve incrementally. The goal is for each developmental cycle to produce a tangible, testable/verifiable outcome that can be demonstrated, tested or used as a basis for further work.

This approach emphasises adaptation over upfront planning, and continuous alignment with the client through a feedback loop over long-term planning aimed at delivering a perfect product in one fell swoop. By doing this, we identify misunderstandings and new constraints early in the process, where the price for refactoring and reworking is lower.

3.2 Iterative Development Model

The iterative model adopted in this project is characterised by repeating the following phases:

1. Requirement Clarification
2. Design and Planning
3. Implementation of a Small Improvement or Feature
4. Validation and Feedback

5. Finding Next Steps

Each cycle results in a working, testable artifact that can be demonstrated to the client as a basis for further exploration and feedback. This ensures that the system converges toward the correct functionality and satisfies requirements.

3.2.1 Requirement Clarification

At the beginning of each development cycle, requirements should be revisited with DBC Digital to ensure that the current understanding of the system aligns with operational realities. In an ideal workflow, this phase would involve:

- Reviewing the previously implemented increment
- Collecting feedback from intended users or infrastructure engineers
- Identifying oversights, misunderstandings, or newly discovered needs

3.2.2 Design and Planning

Before implementing a new feature or architectural element, a lightweight design discussion would take place. In an ideal workflow, this includes:

- Considering different architectural options
- Evaluating trade-offs in terms of security, performance, reliability, and extensibility
- Ensuring that the design aligns with long-term system goals
- Keeping the design minimal so that later cycles can adjust it if needed

The objective is not to create a ton of documentation, but to ensure that what we are designing makes sense.

3.2.3 Implementing a Minimal Increment

The implementation phase focuses on delivering the smallest viable functionality that satisfies the requirements. Examples could be:

- Building an endpoint to receive a specific type of data
- Implementing a prototype scraper or pull mechanism
- Creating a minimal version of the *UI* (User Interface)

In a perfect iterative development world, the increment should be deployable and testable at the end of the cycle, even if incomplete or limited in scope.

3.2.4 Validation and Feedback

Each increment is validated both technically and with respect to client expectations by:

- Functional testing of the increment
- Reviewing architectural soundness
- Verifying that the feature integrates correctly with existing components
- Demonstrating the new functionality to the client and gathering feedback

The purpose of this phase is to detect misunderstandings early and make corrections while changes are still inexpensive.

3.2.5 Finding the Next Step

The final stage is to use the insights gained from validation. In an ideal process, this includes:

- Updating the backlog of remaining tasks
- Re-prioritising features based on technical findings and client needs
- Identifying risks or questions that require further research in the next cycle
- Removing planned low-value features or reducing scope when appropriate

This ensures that the project adjusts to the new incoming information, avoiding costly delays and misunderstandings.

3.3 Advantages of the Iterative Methodology

Using an iterative development model for this project has several benefits:

- Tight feedback loops allow incorrect assumptions to be corrected early
- Client involvement ensures the system evolves in alignment with their needs
- Improved adaptability, allowing architectural decisions to evolve naturally as understanding of client needs evolves
- Early availability of prototypes, enabling real-world testing of the software instead of hypothetical theories

These advantages are significant in infrastructure software, where many constraints only become apparent when interacting with real-world systems.

3.4 Application in this Project

As this was a solo project, the iterative methodology was applied in a not-so-strict manner. I made some decisions based on experience, both past and recent, and through insights that emerged over time in the project rather than through a formal process. The following *Process* chapter describes how this methodology unfolded in practice and where deviations from the idealised model occurred.

4 Process

4.1 Overview

In this chapter, I describe the development process of hostmap. A Gantt chart overview can be seen in Figure 4.1. It is included to show the project as an engineering process, rather than a finished product. It also explains which decisions were made and why. Each section corresponds to one cycle, as described in the previous chapter.

The development happened in two to three-week phases, where each phase ended in a concrete incremental improvement that I demonstrated to DBC Digital and adjusted according to their feedback. DBC Digital let me decide on what process to use, and for reasons mentioned in the previous chapter, I chose an iterative process.

4.2 Early Development — Mid September

The goal of this phase was to establish a collaboration process, collect requirements, and start prototyping.

As explained in Section 5.1.1, all pipelines depend on the Activation Log Pipeline. Because of this, I chose the Activation Log Pipeline (Section 5.1) as the focus for the first phase of development.

The infrastructure team and I agreed that there is great value in on-site collaboration, and therefore, I was given a desk at DBC Digital's office. We made a flexible agreement that I would come in approximately twice a week, depending on the needs of the project, and I will do the rest of the work remotely. I was also invited into DBC's Slack channel, where the team was available for questions throughout the project.

To enable an iterative process, we agreed that project's progress would be shown to the team every two to three weeks. I suggested this to ensure a tight feedback loop, so that any misunderstandings could be corrected early.

To arrive at a working prototype earlier, I reused the activation log files from the old host overview system as a temporary solution. I set up an Nginx server on the three test servers I had been given to experiment with to serve the activation log file that the old hostoverview system produced. This allowed the prototype to function without having completed the Activation Logger.

By the end of this phase, I had implemented functionality that enabled the Scraper to retrieve activation logs one host at a time and parse the CSV into Rust data structures. In addition, the Scraper was able to parse the configuration file containing host

information. Due to time constraints, saving activation logs was not implemented, but I had time to create migrations for the activation log table and host table.

I finished working on an abstraction layer for saving the host information to the database. I decided to save time and make maintenance easier, by letting the Server re-use the Scraper's logic for parsing host information from a configuration file.

I demonstrated the early prototype to show that the Activation Log Pipeline was viable. Based on this, DBC Digital confirmed the direction of the project and indicated they were ready to start working on enabling the CI pipeline to send mappings as soon as the relevant endpoints were close to being available.

4.3 Completing Activation Log Pipeline and First UI — Late September

In this phase, completing the Activation Log Pipeline was prioritised along with saving the data from the pipeline to persistent storage.

In order to improve concurrency for the Scraper, I restructured it to support configurable request intervals and bulk submission of activation data. I re-wrote it to make one request every N seconds, where N is configurable at start-up. These changes allowed a new request to start before the previous one had finished, lowering the upper bound on the time to scrape all hosts.

I also improved error handling so that when an error is encountered, the Server returns the correct HTTP status. This was done to make debugging easier and the code more correct.

Simultaneously, I created a web page for users. The Server now displays data from the database, validating the data flow for the Activation Log Pipeline end-to-end, and taking the first step to implement the View Pipeline.

I had finished prototyping one part of the system, and it was now able to gather activation logs, persist them, and display them to the end-users.

Feedback from the infrastructure team further stated that the work I had done so far met their expectations and that I should continue the work as planned.

4.4 Server and UI Development — Early October

At this point, the structure and maintainability of the Server became the focus. As the codebase grew, business logic had become intermingled with persistence logic. This made reading, testing, and debugging more difficult. I refactored the code to use a service layer between the controllers for the HTTP endpoints and the code persisting to and fetching from the database, which decreased coupling in the code, and followed the standard MVC (Model-View-Controller) pattern.[11]

I also added a historical view per host to the web interface. This view could be accessed by clicking on the respective hosts on the front page. I spent substantial effort and time creating maintainable and efficient code for retrieving historical activation logs for a host from the database. This moved us closer to fulfilling one of the requirements of the historical per-host view.

The historical overview was reviewed by the team at DBC Digital, and they concluded I was on the right track to meet the requirements. In the meeting, I suggested

that we could continue using the legacy system for generating activation logs in order to save time. The proposal was rejected for the following reason: If the legacy system's log generation breaks during the development of hostmap, that would leave the team at DBC completely without a working tool. As a result, the decision was reaffirmed to implement a dedicated Activation Logger.

4.5 Adding Metadata — Early to Mid October

This phase focused on adding metadata to the hosts to enable a generic way of grouping them.

At this moment, the database schema contained a host table that had foreign key references to records in a host group table. This was the model DBC Digital used, and was a reasonable interpretation of the requirements.

As I continued development, and the system was implemented in practice, it became clear that hard-coding the grouping of hosts in this manner limited the hostmap. The modelling I had done was well-suited to DBC Digital's setup, but conflicted with my intention to make the system open-source.

To fix this, I changed the data model. I removed the relation between hosts and a host group in the host group table. Instead, I added a metadata column to the host table, which contained a set of string-to-string key-value pairs for each host. At startup, the Server can be configured with a command-line flag, `--grouping-key`, which determines the key in the set to group the hosts by in the web interface.

This redesign required substantial refactoring of the database schema and the application logic and accounted for a sizeable portion of the use of my time in this iteration. However, the result was a more flexible abstraction that aligns with my long-term goal of open-sourcing the project and makes it easier for others to use.

During this phase, I also created the database migrations for store-path-to-commit mappings, and a lot of time was spent setting up test data, testing, and writing queries that match activation store paths to their corresponding mappings. The most complex of the queries can be seen in Appendix A.

4.6 Usability Improvements — Mid October

In the start of this phase I noticed that identical commits shared the same colour, but the colours could change between page refreshes. This led me to rewriting the method for selecting background colours of the commit hashes. The previous method was replaced with a deterministic approach where a given commit hash always results in the same background colour, in order to make the UI less confusing.

I also fixed several bugs related to host metadata. The biggest one was disallowing nested JSON objects and arrays. I only allowed metadata in the form of string-to-string key-value pairs.

To allow the CI server to send data to the Server securely, I implemented a middleware layer that verifies the validity of the API key when trying to access a protected endpoint. I also spent time writing unit tests for this component early in development to support faster iteration by shortening the development feedback loop.

In addition to these improvements, the functionality for receiving data from the

CI server was completed. I informed the team at DBC Digital that the endpoint was ready, and they indicated that they were close to finishing their implementation for sending data from the CI server to Hostmap.

4.6.1 Timing-Attack Mitigation

```
1 // before - vulnerable to timing differences
2 client_api_key == server_api_key;
3
4 // after - constant-time comparison
5 bool::from(client_api_key.as_bytes().ct_eq(server_api_key.
    as_bytes()))
```

Figure 4.2: Replacing string comparison with a constant-time comparison to avoid timing-based side channels.

While writing unit tests for API key validation, I realised that the comparison logic was vulnerable to side-channel attacks [12], as shown in Figure 4.2. This issue was not detected by a failing test, but rather the process of writing the tests revealed the potential risk. As a result, I replaced the string comparison with a constant-time comparison function to mitigate timing attacks.

4.7 CI Server Integration — Mid October to Late November

During this period, the endpoint for receiving CI mappings was first tested using simulated data. After minor bug fixes, the functionality appeared to work as expected.

Once DBC Digital finished implementing the code for sending mappings, the integration was tested. This process took longer than expected due to scheduling and coordination issues. In the first integration attempt there was a mismatch in what we expected the endpoint name to be. This took some time to resolve. During earlier discussions, multiple alternative names for the mapping endpoint had been considered. After the error had been corrected, additional issues were discovered in the incoming data, which required adjustments on DBC Digital's side.

After these last errors were fixed, we essentially had a usable version of hostmap in place. We agreed that the next step was to implement the Activation Logger and thereafter deploy the system for testing.

4.8 Project Slowdown — Late October to Mid November

In late October, due to unforeseen personal circumstances, development progress stalled for several weeks. This represented a significant deviation from the planned two- to three-week iterative cycles. As soon as the delay was recognised, DTU was

contacted and approved a three-week extension to the project deadline. I also explained the situation to DBC Digital, and they were understanding. This essentially had the effect of postponing the iteration cycle I had planned by three weeks.

4.9 Final Code — Mid November

Once I resumed working, the project entered its final implementation phase. I built the Activation Logger and deployed it to my three test servers. Due to firewall issues at DBC Digital, the development and deployment of the Activation Logger took longer than expected. Serving traffic over a specific port worked when using Nginx, but failed when using the custom-built Activation Logger directly.

I had difficulty debugging these issues, as I believed the fault to be in my code. I was convinced that since the Server can fetch the data over that port when using the Nginx solution, and it doesn't work with my custom solution, something must be wrong on my end.

After discussing the issue with my mentor from the infrastructure team, he investigated further, identified the root cause, and resolved the firewall issues.

At this point, the project was completely self-contained and no longer relied on any components of the old system.

4.10 Deployment to Test — End of November

During this phase, I made the necessary Nix configuration changes to DBC Digital's Nix monorepo to deploy the Activation Logger to approximately 40 test servers. The deployment succeeded, and there were no immediate bugs. Both the team at DBC Digital and I agreed that we should let the new system run for a week on the test and staging servers. If everything worked, it would be deployed to production in the following week.

Additionally, to let the server scrape on a shorter and more precise schedule, I modified the Scraper to batch HTTP requests instead of sending them one at a time. Requests were now sent in batches on a timer rather than individually. I also implemented a catch-up mechanism in the Scraper to ensure stable scrape intervals even when individual batches took longer than expected, further described in Section 5.4.

I also added a search feature for the front page, along with several cosmetic CSS improvements as part of the final deliverable, and I also completed one of the stretch goals by enabling `nix-diff` functionality, a command for diffing two Nix store paths. These features are described in more detail in Section 5.5.3.

4.11 Deployment to Production — Start of December

After one week of showing no errors in the test and staging environments, `hostmap` was deployed to production.

At the time of writing, the system has been running for several weeks without issues and is used daily by the infrastructure team, replacing the use of the old system in their workflow. The legacy system is still running, but is planned to be

removed.

4.12 Conclusion

The methodology described in Chapter 3 served as a guide, but the actual process required more flexibility to make room for some constraints, such as integration of dependencies and unpredictable delays.

Even though there were deviations from the ideal iterative model, the approach worked really well. Early prototypes allowed for feedback from the team from the start, and helped influence later decisions, such as the redesign of how to group hosts to use a metadata column. I particularly valued the close collaboration with the infrastructure team, especially their help on Nix-related challenges in the project.

This process resulted in a complete and well-functioning system that had improved over multiple iterative cycles and culminated in deployment to hundreds of production servers.

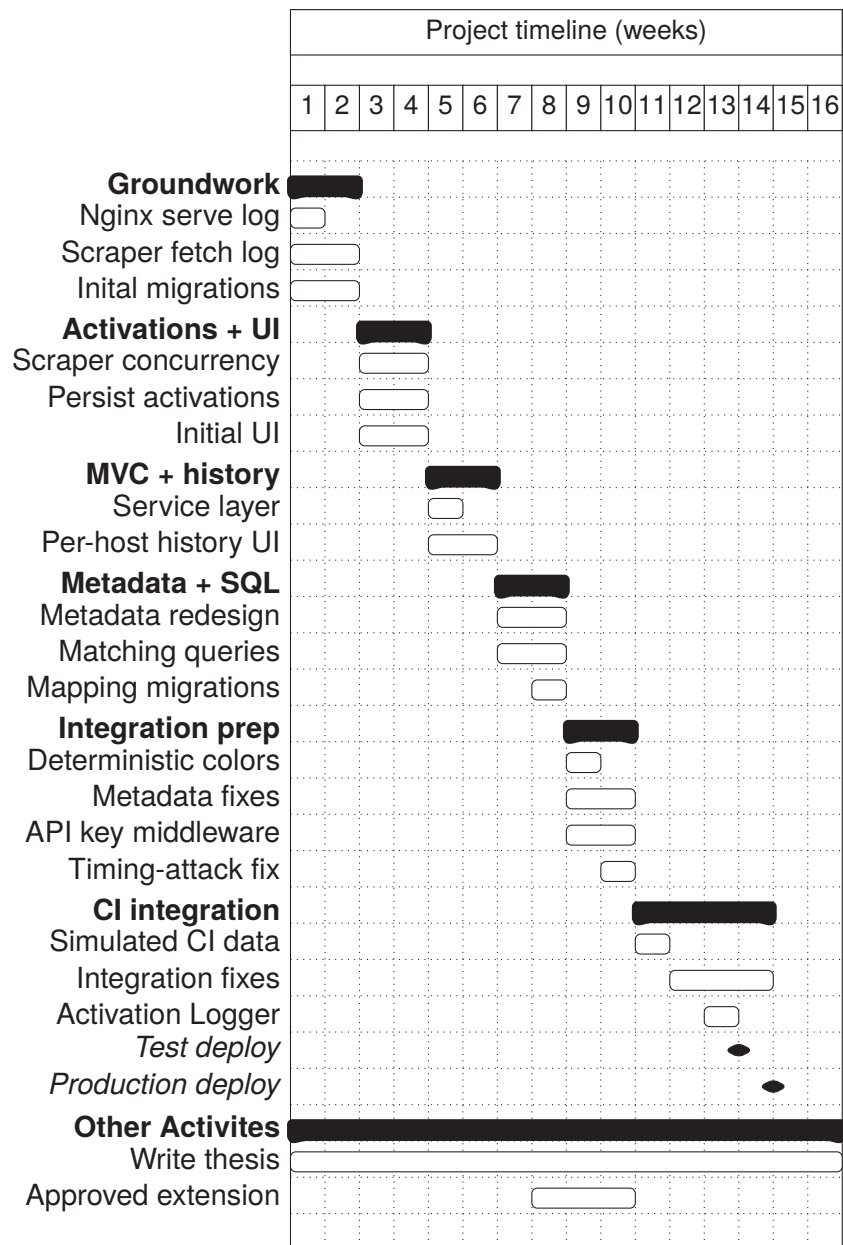


Figure 4.1: Timeline of project process. Reduced activity during weeks 8–10 due to unforeseen personal circumstances.

5 Solution

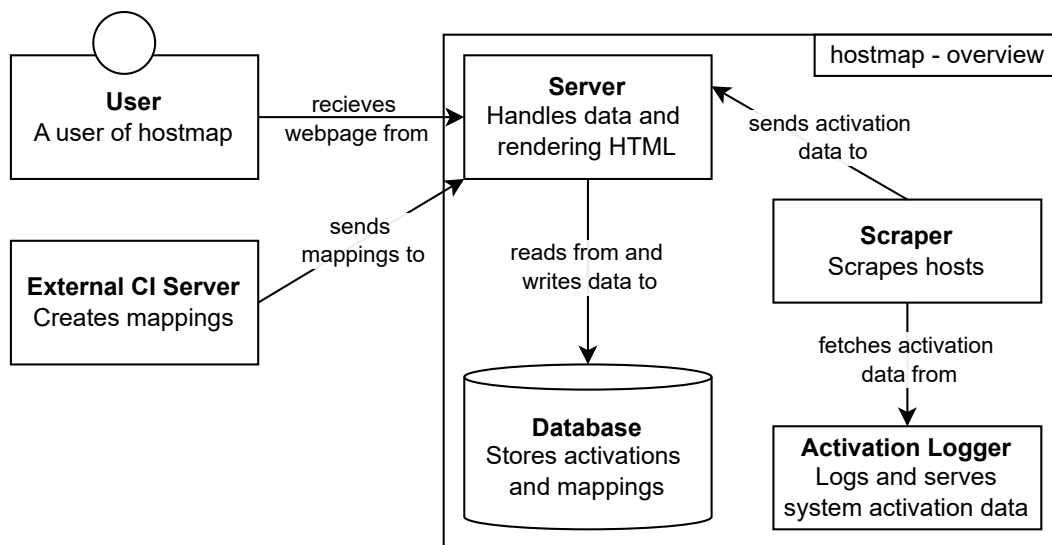


Figure 5.1: An overview of hostmap's components and their interactions

This chapter describes the final technical solution. It explains the flow of data between hostmap components, the architecture, and the implementation choices. I include this chapter to make the system easy to understand and to show how I chose to complete the requirements described in Chapter 2.

The final hostmap solution is a system that tracks the NixOS system image state of hosts and ties them to the Git revision they originated from. The architecture consists of three components that work together: the Scraper, the Server, and the Activation Logger, as shown in Figure 5.1.

5.1 Data Flow

Each component has a well-defined role, and all communication between them is over HTTP. At a high level, the system consists of two incoming data flows into the Server's database and one outgoing user-facing data flow, as seen in Figure 5.2. A summary of the data flows is as follows:

The first incoming data flow, referred to as the *Activation Log Pipeline*, describes the path that runtime activation logs take from each host to the back end. Each NixOS host runs the *Activation Logger*, which exposes the activation log over HTTP. The *Scraper* periodically scrapes a list of hosts from a configuration file, by default

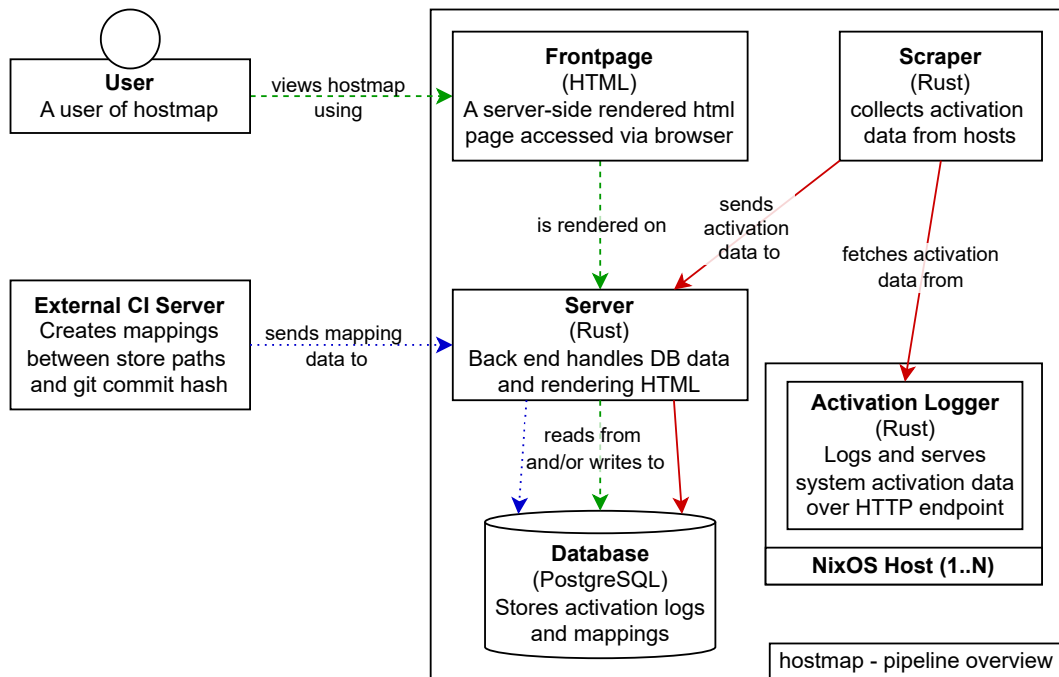


Figure 5.2: The data flows in hostmap. Activation Log Pipeline (red, solid), Mapping Pipeline (blue, dotted), View Pipeline (green, dashed)

in batches of eight, with 15-second intervals between the batches. Having fetched the activation logs from a batch of hosts, the *Scraper* posts the data to the *Server*, which stores it in a PostgreSQL database.

The second incoming data flow, referred to as the *Mapping Pipeline*, describes the flow of store-path-to-commit (and branch) mappings from the CI server to the back end. The back end receives a POST request with mapping data. Generating these mappings is out of scope for this project. In DBC Digital's case, their CI server creates this mapping and posts it to the back end after the build, and then stores the mappings in the same PostgreSQL database as the activation data.

The outgoing data flow, referred to as the *View Pipeline*, describes how store-path-to-commit (and branch) mappings and activation data are retrieved from the database and shown to the user. In this flow, the back end joins hosts and activation logs with the corresponding mapping data and serves the result as an HTML page.

5.1.1 Pipeline Dependencies

The *Activation Log Pipeline* describes the path that activation logs take from the hosts to the back-end database. This pipeline is foundational to the system. Without activation logs, we cannot determine which store path a NixOS host is currently running, meaning we cannot use the store-path-to-commit-hash mappings provided by the *Mapping Pipeline* meaningfully. Similarly, without activation logs, the *View Pipeline* cannot show users meaningful information about the hosts. The system could only show static information, such as the hostname and the group they belong to, based on metadata grouping keys.

5.2 Architecture and Technology Choices

First, the overall system architecture was designed, and afterwards suitable technologies were selected.

5.2.1 Architecture

There were two important architectural decisions regarding what method to use for moving data to the back end.

When fetching data from the server fleet to the back end, we selected a pull-based approach. This enabled us to have a basic HTTP server serving a log file as the only thing the hosts needed to do. If we were to choose a push-based approach, the logic running on the hosts would have to be more advanced. For example, the servers in the fleet would individually need to be able to detect when a request had silently failed and run retry logic.

Moving to a pull-based approach means that only the back end will need to have this functionality, instead of hundreds of servers.

In addition, using a pull-based approach allows the back end decide on the load. If the approach is push-based, hundreds of servers could potentially make POST requests at once to the back end, subsequently overwhelming it.

We also avoid the need for authentication. The back end simply fetches the data without authentication, because the Activation Logger only serves non-sensitive information. With a push-based approach, we would need authentication so that malicious actors cannot send junk data, so we avoid the need for handling and rotating API keys for hundreds of servers.

When moving data from the CI server to the back end, a push-based approach was selected. Several of the problems mentioned above are not relevant or diminished because there is only one CI server, so there is a much smaller set of API keys to handle. Additionally, using only one server to send data makes it unlikely that the back end receiving the mappings will be overwhelmed.

We also gain a significant thing. Since the CI server knows exactly when it has built the store path from the git commit, it can send the data immediately to the back end, meaning that the data will be fresher.

5.2.2 Technology Choices

The code in the hostmap repo is written in Rust. A few other alternatives were considered. I decided not to use Java because Nix is significantly easier to work with when using self-contained binaries. Using Java, I would have to ship the JVM alongside it. I also knew that parts of the code would be running on hundreds of servers, so I really wanted to at least write the *Activation Logger* in a language that compiles to a binary. Using async code was an easy decision. Async has been a part of Rust for quite some time now and is very stable. It can make some things harder and give you very cryptic error messages, but besides that, it is very convenient. I also wanted to write all code in Nix and one more language, and in a language that the team at DBC already know, so it would make maintenance easier for DBC Digital once I leave.

I have previously experimented with a few of the Rust web frameworks, and

Axum has been the most pleasant to work with. Axum was also chosen because the project maintainers have a strong focus on correctness and ergonomics, which contributed positively to the long-term maintainability of the solution, and ease of use.

I settled on using a relational database because the data I was going to work with naturally lent itself to relational modelling. PostgreSQL was chosen over alternatives such as MySQL and SQLite because it is the safe, industry-standard default, and because I have prior experience with it. In meetings with the infrastructure team, I learnt that they use PostgreSQL in production, which further convinced me to choose it.

When selecting the database technology, the requirement for storing arbitrary host metadata was not yet well understood. As the requirement became clearer, the fact that PostgreSQL supports JSON and JSONB[13] made it easy to extend the schema to include metadata, with only small changes to the rest of the schema. Although the JSON support was not a factor when deciding on the technology, its availability is the reason why we chose to store the metadata as JSON in the host table.

I elected to server-side render all the HTML. I did this because, from the requirements, I knew that the application would not need a lot of client-side user interaction. The web interface is almost exclusively read-only tabular content. Using a SPA (Single-page Application) framework such as React would add a lot of dependencies for little to no gain. For server-side rendering, Tera was used as the templating engine. Many of the templating languages in the Rust ecosystem embed the templates in the binary. This makes deployment easier. One of the aims for hostmap was to make it open source for others to use. Since Tera doesn't embed the templates in the binary, I thought using Tera might make it easier for other users of the project to use their own templates if they wanted to change how the web interface looks. If I used one of the many solutions that embed the templates in the binary, users of hostmap would have to rebuild the Server to see their custom templates.

Supporting libraries and Data Formats

In addition to the core technologies above, a few smaller technologies are worth mentioning. For compile-time checked queries in Rust and migrations, I used *sqlx* and *sqlx-cli*. *Sqlx* gives a compile-time error if the code has SQL that doesn't match the tables or columns in the database [14]. *Sqlx-cli* can be used to generate and run migrations [15]. This means that when the code compiles, I won't get runtime errors, which is a huge advantage. Using *Sqlx* over an ORM (Object Relational Mapping) tool means I get precise control over the queries, and I have access to PostgreSQL-specific features. I made heavy use of several PostgreSQL features, such as `DISTINCT ON`, JSON support and CTEs (Common Table Expression), which means an ORM tool would just have gotten in the way.

I use Tokio as the async runtime for all the Rust code. It integrates great with Axum and makes writing concurrent code straightforward.

5.3 Activation Logger

Every host that we want to be scraped needs to have the activation logger running. Enabling the activation logger in the hostmap Nix module does two things. It enables logging of Nix activations, and it exposes the activation data.

The logging of Nix activations is done in the following way. As I described in Section 2.2, system activations are made through the `switch-to-configuration` script. When rebuilding the system image, the original `switch-to-configuration` is moved to `_switch-to-configuration` and a wrapper script is added under the original name. The wrapper script, when run, logs the following activation data to a CSV file: timestamp, current user, store path, and Nix activation type. It then executes the original script. Every activation will now be logged. Any time a system image is switched, the wrapper script is executed first before invoking the original script. This logging mechanism is adapted from the legacy system used at DBC Digital. The adapted logging mechanics were modified to be compatible with the legacy system by supporting and migrating the old logs.

The other part of the Activation Logger is a small HTTP server in Rust and Axum that serves the logs over HTTP. The HTTP server is intentionally designed to be as minimal as possible. It doesn't compute anything; it just serves the data from the activations log file to anyone requesting it. It sets appropriate headers to prevent caching, so we ensure that the client fetching the data gets the latest content and sets headers that identify the content as CSV text. The simplicity and lack of computation here ensure low resource usage on the hundreds of hosts on which the code is deployed. Since the data it serves is not sensitive, and the endpoint is read-only, there is no need for authentication, which is one advantage of using a pull-based approach.

5.4 Scraper

The Scraper is responsible for moving the data from the host through the endpoint exposed by the Activation Logger to the Server. It finds which hosts to scrape by reading a configuration file that contains, for each host to scrape, hostname, host url and metadata. When reading the configuration on startup, it tells the Server what hosts will be scraped. This ensures that the Server has up-to-date information about all hosts. All communication between Scraper and Server is done over HTTP, and through API-key authentication. After the initial registration of hosts, the scraper enters its main loop. In batches, the scraper on a timer makes concurrent requests to the Activation Logger endpoint to retrieve the CSV log. If the scraper is configured to fetch a batch of 8 logs every 10 seconds, and there are 240 hosts, it is expected to fetch all hosts in:

$$\frac{240}{8} \cdot \frac{1}{6} = 5 \text{ minutes}$$

This means that the worst-case time for this configuration is 5 minutes. If the timer falls behind, e.g. because of a failing request, or because of the system being slow to parse the CSV file, it is configured to try to catch up to stay on track, so the system makes a best effort attempt to ensure that the worst-case scenario is never

worse than 5 minutes in the example above.

To do this, the Scraper uses a fixed-rate scheduling strategy. It aims to start a batch of requests every N seconds, instead of sleeping for a fixed duration. When it has completed a batch, the Scraper calculates the time left until the next batch should start and only sleeps for that duration. If a batch finishes later than the next scheduled start, the sleep is skipped, and allows for the Scraper to gradually catch up, so it does not drift further behind the schedule. This prevents delays from accumulating and ensures that the total time required to scrape all hosts is close to the expected value $N \cdot M$, where M is the number of batches.

The Scraper is not responsible for persisting the activation data and is stateless with respect to it. All activation data is sent to the Server for storage. Because of this, multiple Scraper instances can submit activation data to the same Server at the same time.

5.5 Server

It is responsible for saving the data from the Activation Log Pipeline and the Mapping Pipeline, and combining the saved data in the View Pipeline to display to the user.

Therefore, architecturally, the Server is the point where the three pipelines, described in Section 5.1.1, join. This means it is the Server's responsibility to coordinate them. Apart from the Activation Logger keeping a local log on a host, this is the only component that stores state over time. In the database, it defines the models and the rules for querying, changing, and inserting host data.

It is responsible for four things:

1. Storing the activation logs received from the Scraper
2. Storing the store-path-to-commit mappings received from the CI server
3. Correlate the host's activation data with the mapping data when the user queries it
4. Present the correlated data in a web interface to the user

The server does *not* scrape hosts or generate CI mappings. What it does instead is expose endpoints that other components can send data to.

5.5.1 API Design

There are two types of endpoints. Public endpoints which are used by system users to access the web interface. Protected endpoints that other machines use to send data to the Server.

The protected endpoints include bulk insertion of hosts, activations and mapping data. These endpoints require authentication through a shared API key. The client sends the key using the following HTTP header: `Authorization: Api-Key <api-key-token-here>`. It uses constant-time comparisons to avoid side-channel timing attacks as described in Section 4.6.1. I implemented the authentication as middleware, so any unauthorised requests are rejected immediately, before any application logic happens.

5.5.2 Database

A PostgreSQL database is used for the data. For potentially write-heavy queries, e.g. activations and mappings, the controllers and repositories have both single and bulk endpoints for insertions. In addition, all insertions are idempotent. If a store-path-to-commit-hash-mapping that already has an identical row in the database is inserted, it is not duplicated. This means that retries due to network failures or if the scraper restarts will not create duplicate rows.

The most complex SQL query in the Server is matching each host and latest activation with the correct Git revision and branch (see Appendix A). It must select the most recent activation for each host, join it with all commit mappings for the activation's store path. Among these mappings, the query selects the row with the earliest `linked_at` timestamp, but prioritises rows with a branch called "master" over rows from other branches. How correctness was evaluated and verified for this query is explained in Section 6.1.2.

5.5.3 Web Interface

In hostmap's Server component, the correlation between activation data and mapping data happens when a user requests the web page. If there is no mapping yet for the store path, a placeholder with "N/A" is shown for commit hash and branch. The Server renders all HTML pages on the server using the Tera templating engine. This fits the requirements, since the UI is mostly read-only and mainly shows tabular data.

The Server uses Server-side rendering, and that has some advantages:

- Less complicated logic client-side
- Easier deployment, since there is no separate frontend build pipeline.

The pages focus on presenting hosts, activations, and Git revisions in a way that makes it easy for operators to check the current cluster state and historical changes.

To further improve the UX (User Experience), there are some enhancements to the web interface using JavaScript. On the front page, the user can search, as seen in Figure 5.3. The rows are searchable by hostname, metadata, branch, and store path.

In the historical overview, in addition to seeing the historical store paths and their corresponding git mappings, the user can select two store paths and get a command to run in the terminal. That command runs `nix-diff`, a command-line tool that shows the difference between two nix derivations, which DBC Digital had requested as an optional stretch goal.

5.6 Nix Integration

An integral part of the solution is that it integrates tightly with Nix and NixOS. The full system is packaged and deployed using declarative Nix expressions, so development and deployment are reproducible.

The project is distributed as a Nix flake that defines:

- A reproducible Rust build of the hostmap binary

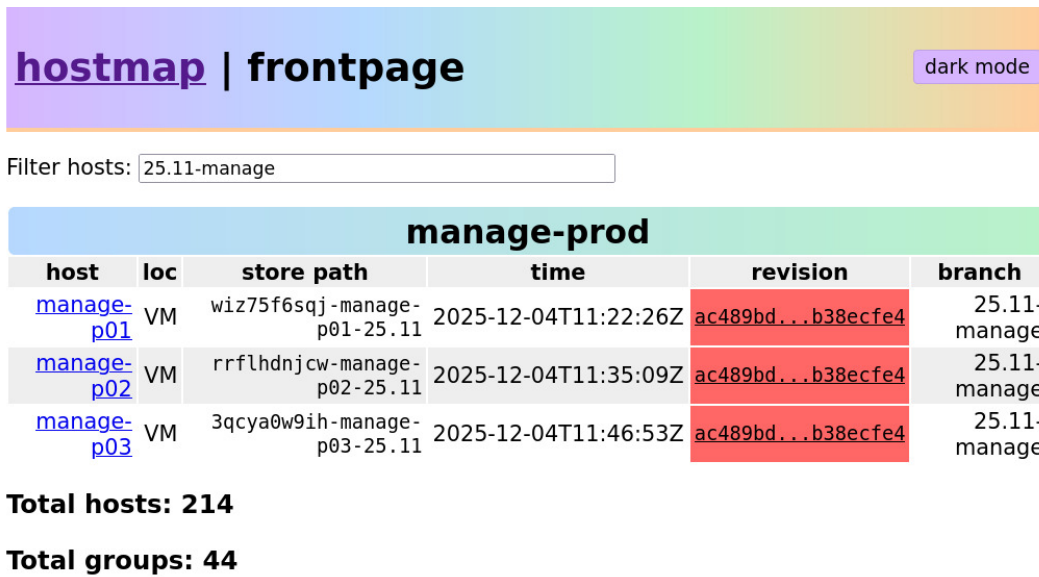


Figure 5.3: Filtering on hosts by a branch name (data has been truncated for brevity)

- A development shell with the required tools for running it
- A NixOS module for deploying hostmap as system services

All important build inputs, such as SQL migrations and HTML templates, are included in the derivation. This means that the same Git commit will build the same runtime artifacts.

5.7 Hostmap Deployed at DBC Digital

In the deployment of the Server and the Scraper at DBC Digital, three servers are used. Each server runs one instance of the Server and one of the Scraper. The Activation Logger runs on every monitored host in the fleet, including the three servers running the Server and Scraper components, in total on over 200 servers.

Since there is no coordination between the PostgreSQL instances, the data can become temporarily inconsistent. The team prefers this simplicity over guarantees of consistency for their use case and finds the temporary inconsistencies acceptable as a worthwhile trade-off.

5.8 Summary

Hostmap was implemented as three Rust-based services deployed using Nix. The only external runtime dependency is the set of Tera templates, which are packaged through the Nix configuration, so there is no need for users to manually provide them. These choices led to a system that is easy to build, configure, and deploy, which is almost entirely a self-contained binary.

nix-diff command

nix-diff command complete. copy to use it.

```
# Host: hosts-p01, Commit: 7a3812e9..fd86107f
DRV_A=$(nix eval --raw "git+file://$PWD?rev=7a3812e90f0f0cb8755061cd197ab0df8406d06f#nixosConfigurations.hosts-p01.config.system.build.toplevel.drvPath")
DRV_B=$(nix eval --raw "git+file://$PWD?rev=fd86107f4dc817718f25b868cd4c980e594e972c#nixosConfigurations.hosts-p01.config.system.build.toplevel.drvPath")
nix run nixpkgs#nix-diff -- "$DRV_B" "$DRV_A"
```

Copy nix-diff command

activation logs

2025-12-08						
time	user	system	action	commit hash	branch	nix-diff
07:45:48	srhb	xndnwpw5fc-hosts-p01-25.11	boot	7a38...d06f	master	☒ Δ
07:37:31	srhb	7xdqhyr01r-hosts-p01-25.05	boot	9582...8b74	master	☐ Δ
2025-12-04						
time	user	system	action	commit hash	branch	nix-diff
20:53:32	exheze	7xdqhyr01r-hosts-p01-25.05	boot	9582...8b74	master	☐ Δ
20:43:50	exheze	4m0v9chrln-hosts-p01-25.05	boot	fd86...972c	move-module-to-hostmap-repo	☒ Δ
20:40:37	exheze	4m0v9chrln-hosts-p01-25.05	boot	fd86...972c	move-module-to-hostmap-repo	☐ Δ

- [hostmap repo](#)

Figure 5.4: Selecting two rows to get a nix-diff command (data and headers have been removed for brevity)

6 Evaluation

This chapter evaluates whether hostmap satisfies the system requirements and whether it actually improves on the legacy system. It is also included here as evidence to prove that the design decisions from the previous chapter are justified.

The project was tested using a combination of local testing, integration testing against test servers for activation log fetching, testing the correctness of matching mappings to fetched logs, and end-user tests.

6.1 Testing and Validation

To make sure that the system fulfils the requirements, a series of tests was conducted at different levels for different parts of the system.

6.1.1 Unit Tests

Unit tests were written for selected parts of the core of the system. Most of the unit tests focused on correctly implementing the authentication of API keys. These tests included handling malformed headers in authentication requests, incorrect prefixes, and unexpected white-space.

While writing these tests, a vulnerability was discovered. The API key comparisons were open to side-channel attacks, since they used string equality comparison instead of constant-time equality comparison. Although this was not discovered through a test failure, the process of writing the tests made the issue apparent. The comparison was changed to use a constant-time comparison, which improved security in the final product and aligned with one of the non-functional requirements.

Some non-critical components were also tested, and here I wrote the tests before writing the implementation of the method. This was done for the method that shortens the system paths into easier-to-read text by removing some of the hash in the middle.

I also wrote unit tests for conversion between errors to ensure that the correct HTTP status code is returned upon error.

These methods all have in common that they are pure, and that is why they were selected for unit testing. Due to time constraints and due to the conversions from DTO (Data Transfer Object) to model being trivial, I elected not to write unit tests for this mapping.

hostmap

[dark mode](#)

Note: The **rev** column displays the *earliest known* (from CI) revision that produces a given system path. That means that: Given two revs; A and B, which both produce the same system path, and of which A is the earliest known (that is, built by CI), an "update" from A to B will *not* result in the rev column being updated.

Filter hosts:

test				
host	store path	time	revision	branch
test-1	/nix/store/efgh5678	2023-03-01T11:30:00Z	3CORRECT	master
test-2	/nix/store/ndkj2312	2023-03-01T10:10:00Z	4444cccc	master
test-3	/nix/store/ijkl9012	2023-03-01T09:40:00Z	3333cccc	master

Total hosts: 3

Total groups: 1

Figure 6.1: Matching host's current store path (yellow), with the correct git commit and branch (blue)

6.1.2 Local Integration Tests

I wrote local integration tests for the most complex SQL in the project, shown in Appendix A. It matches the latest activation per host to the correct Git revision and branch, and I describe it in more detail in Section 5.5.2.

Since this logic involves ordering rows multiple times on multiple arguments and has a non-trivial JOIN, and since unit tests generally are never enough to test SQL queries, correctness needs to be validated through integration tests.

The integration test was created by writing a carefully constructed dummy data seeding script. I designed the seed data manually, thought through what the expected behaviour should be, and then explicitly marked the correct and wrong commit hashes of the mapping. For the entry with hostname *test-1*, the correct hash, that should display, was marked *3CORRECT*, whereas the other two, a later master branch commit, and an earlier commit from a branch not called master, were called '00latest' and '555WRONG'. This made it immediately clear when testing whether the system had mapped correctly.

After seeding the database, the back end part of the system was started, and then I could use the UI to confirm that the expected commits appeared in the view, as shown in Figure 6.1.

This test also verified the requirement that the system can detect multiple Git commits that map to one store path.

6.1.3 Mapping Endpoints Simulation

To test the endpoints on the Server responsible for receiving commit-to-store-path mappings, without relying on the production CI server, a simulation was performed.

Relevant test data was created by using existing production mappings as a template. This ensured that the structure and content of the data matched what the system would receive in a real deployment. The endpoint was then tested by using *curl* to send POST requests with JSON payloads containing the mappings.

This test was done to verify that the Server correctly handled incoming mapping data, including validation of authentication credentials, handling of malformed requests, and correct persistence of valid mappings in the database.

By simulating the CI server like this, the logic of the mapping endpoint and persistence of the data by the Server could be tested in isolation, without relying on the external CI server. The test was considered successful when the received mappings appeared correctly in the database and were visible through the web interface as expected.

6.1.4 Integrating CI-Server and Load Testing

To test the integration between the system and DBC Digital's CI server, a test was conducted using production mapping data.

In this test, all mappings up to a month before the test were sent; no newly generated mappings were sent. This means that the system received millions of mappings from the CI server over a span of an hour. As a result, the test functioned both as an integration test and as a performance test.

The purpose of this test was to verify that the *Server* correctly received mapping data, and that the integration between CI-server and hostmap back end worked. It also tested that the system stores the data consistently in the database and can handle mappings received in arbitrary order.

The test was verified in the same way as the previous mapping tests, by confirming that the mappings were present in the database, with values we expect, and by using the web interface to verify that the correct commits were displayed.

During this test, mappings were sent at a rate several orders of magnitude higher than what is expected during normal CI operation, which made us more certain that we had a wide enough safety margin in this part of the system. In addition, reliability was also tested. The idempotency of the endpoint receiving the mappings was tested by sending the same data multiple times.

6.2 Manual End-to-End Testing

Certain functional requirements concerning the system behaviour are best validated at the complete application level, rather than through unit or integration tests. Therefore, the following requirements were verified through manual end-to-end tests using the web interface.

I verified that the system correctly presented an overview of all hosts with their respective current Nix store path and the Git revision it originated from. I also verified that the system allowed for viewing the historical relationship between activations, store paths, and revisions per host.

The requirement that the system allows for grouping hosts by a configurable key was also validated similarly. The server was started with different values for the `--grouping-key`, and through the web interface, I confirmed that the hosts were grouped according to the grouping key that had been selected.

6.3 Security

The biggest security improvement in hostmap compared to the legacy system is the removal of SSH-based access to hosts, replacing it with a minimal, read-only HTTP endpoint.

Evaluating this was done through discussions with members of the DBC Digital infrastructure team. The lead infrastructure developer at DBC wrote the following:

det har en meget sikrere snitflade (simpel, isoleret HTTP-scrape frem for en, om end meget upriviligeret, bred SSH-adgang på tværs af hosts)

6.4 Data Freshness

In the legacy solution, activation logs are collected by a CI job scheduled every 15 minutes. Since the job itself takes approximately 10 minutes to complete, the worst-case delay before a new activation is visible is approximately 25 minutes.

In contrast, hostmap's scraper polls hosts continuously in configurable batches. With the configuration used during evaluation, the worst-case delay was approximately 5 minutes, representing a five-fold improvement in data freshness.

Also, in feedback, the developers at DBC Digital highlighted the improved freshness of the data. This improvement was mentioned even though the system is running the conservative default settings, scraping 8 hosts every 15 seconds. If needed, the infrastructure team can increase the number of concurrent requests and reduce the scrape interval, at the cost of increased network activity. The precise choice of scrape intervals and concurrency levels beyond the default configuration is left to users of hostmap and is outside the scope of this project. Both analysis and user feedback confirm that the data freshness is significantly improved compared to the legacy solution.

6.5 User Evaluation

6.5.1 Feedback from Infrastructure Developers

Sarah Brofeldt (Lead Infrastructure Developer) and Jonathan Zielinski (Infrastructure Developer) both mentioned that the previous system needed modernisation and that the new system offers clear advantages. The feedback was given both in conversation and in written form. For transparency, the written statements are included in Appendix B.

Sarah mentioned three areas where she noticed improvements.

- The system is decoupled from the CI server, meaning the hostmap servers can easily be spun up or shut down.

- It is HA/redundant. The system will keep working even if one or several of the machines running it crash.
- It has a more secure interface, the SSH access has been replaced with HTTP requests

She also noted that the GUI (Graphical User Interface) improvements, in particular the search functionality, had been much more useful than expected. When upgrading the server fleet from NixOS 25.05 to NixOS 25.11, she used the search feature to filter for hosts that had not been updated yet. Lastly, she noted the much-improved update frequency as a big usability gain.

In a similar vein, Jonathan reported that the new system has completely replaced the old in his day-to-day workflow. The main reasons being:

- The new search feature, which has made locating specific hosts easier
- The improved update frequency of the data, so the UI reflects the actual systems the servers are running

6.5.2 Summary

In all, the user feedback indicates that the system meets all requirements, in addition to satisfying the practical needs of DBC Digital's infrastructure team. The fact that the new hostmap tool has supplanted the old tool in a matter of weeks since deployment to production indicates that the project has delivered real and tangible value to DBC Digital.

7 Discussion

This chapter interprets the results of the previous chapters to place the project in a broader context. It also discusses design decisions and trade-offs that I made during development, and the strengths and limitations that hostmap has. It also examines future work to be done in relation to the project.

7.1 Why Hostmap Fills a Gap

There are systems for tracking the build provenance of a system, among them Nix and SLSA. Other tools, such as GitOps, focus on what we want a system to converge on, see Section 2.4.5. Very few tools focus on what is actually running on a host at a given point in time and when that changed. When we examine these provenance and deployment tracking systems, a gap becomes apparent.

Neither of these two approaches gives visibility into the actual system configuration state of individual hosts across time. SLSA, for example, records how an artifact was produced but stops after it is built, and does not track when, where and how the artifact is deployed. Conversely, systems such as GitOps manage the desired configuration state across hosts in a server fleet, tracked in Git revisions, but track the desired state primarily instead of recording the actual configuration active on a host at a specific time.

One potential reason why few systems fill this role is that it is difficult in practice. Often, systems are not identified by cryptographic hashes, but instead by version numbers, tags or other mutable identifiers rather than unique cryptographic hashes derived from the content of the system. For such systems, it is difficult to determine whether two systems are running system configurations that are bit-by-bit identical.

Though a system such as SLSA can provide these strong cryptographic guarantees, it does not define a mechanism for tracking when artifacts are deployed on individual hosts. This is made even more difficult because most deployment environments lack a single well-defined entry point through which all changes to the system configuration state are made.

NixOS is different. All changes of system configuration state, or system activations in Nix terms, go through the `switch-to-configuration` script, as described in Section 2.2. This enables observing all system activations by hooking into this script. This reliable and simple method of recording activations and corresponding store path enables logging the event, without changing any other tools.

Therefore, hostmap can capture runtime activation data in a very simple way, something that would be much more challenging in other environments. SLSA could, in principle, be extended to include runtime activation tracking, but it lacks

both easy logging of activations and a stable system identifier, similar to a Nix store path, as part of the system at run-time. Implementing similar functionality on top of SLSA would require much more tooling on every host, and coordinating it would be more complex. Utilising NixOS' activation model and store paths, hostmap provides this information in a straightforward way.

7.2 Strengths of Hostmap

Hostmap improves the legacy *hostoverview* workflow by separating concerns and by removing the CI server from the role as data collector and from showing the web interface. In the old system, the CI server both built the system images and also scraped activation logs over SSH, put the data together, and showed the data in a web page. The old system created tight coupling and risk. Decoupling this from the CI server makes the IT infrastructure as a whole easier to understand, and it makes each component responsible for one clear task.

By the host serving a file, using the Activation Logger, instead of requiring SSH access, security improvements naturally emerge as a feature of the architecture of the solution. When SSH access from a central machine is no longer required for the production fleet, the risks disappear. The Activation Logger is kept deliberately minimal and read-only. It serves a single CSV file over HTTP and performs no computation, which reduces complexity and attack surface.

Another strength is improved data freshness. The scraper's timer-driven polling model yields better update frequency than the legacy "build first, scrape later" approach. In practice, this means that the UI can reflect changes in the server fleet within minutes instead of up to 25 minutes, which makes the tool more useful for monitoring.

Hostmap also benefits from being designed for maintainable operation. Packaging the components with Nix makes deployment predictable, and the system is close to self-contained at runtime since it is compiled to native binaries, and required runtime assets are packaged through the Nix configuration. This reduces the number of moving parts during operation and makes it easier to redeploy or add new instances when needed.

One feature that is possible due to the decoupled architecture is that the scraper is horizontally scalable. The scraper is stateless with respect to the activation data and sends it over HTTP. Because of this, multiple scraper instances can submit activation data to the same server at the same time.

To enable this, each scraper would be deployed independently and configured with a subset of the hosts to be scraped. This allows large server fleets to be partitioned across multiple scrapers if needed.

Lastly, the solution is extensible in a practical way. The decision to store host metadata as key-value pairs and make grouping configurable means the system fits DBC Digital's needs while still being useful for other organisations with different grouping semantics. This was a direct outcome of the data model change described in the Chapter 4.5.

7.3 Trade-offs and Limitations

Several design choices in hostmap intentionally trade strictness for simplicity and robustness.

First, the pull-based ingestion model means that data freshness is limited by the scraper schedule. Even though this is a large improvement compared to the old workflow, the scraper is always a little behind: a host can activate but not yet be scraped. This trade-off was acceptable because the expected activation frequency is low on average, and because the pull-based approach avoids both authentication management and the risk of the back end being overloaded from too many push events at once.

Second, the Activation Logger endpoint is intentionally unauthenticated. This is justifiable because the served activation data is not considered sensitive and the endpoint is read-only, but it also means that the deployment relies on network segmentation and firewall configuration. In other environments where the host endpoints are more exposed, an attacker could fetch activation logs or infer deployment timing. For the intended environment, this is still an acceptable risk, but it remains to be investigated if it is risk-free in all environments. Additionally, the Activation Logger does not perform log rotation. Since DBC's hosts are expected to activate at most once per day on average, the activation log grows slowly and remains short. As a result, log rotation was not required to meet the project's requirements.

Third, hostmap relies on the NixOS activation process. This process has remained consistent in practice, but because hostmap depends on it, changes in NixOS could require minor adjustments to hostmap.

Fourth, hostmap assumes the activation logs and mapping data it receives are trustworthy. It focuses on recording and correlating the data it receives, because the problem it aims to solve is providing observability into trusted provenance inputs rather than verification of untrusted provenance inputs. As a result, it does not handle the generation of mappings. This can be viewed as a limitation, but it is also a conscious design decision, aligned with the client's wishes and the scope of the project.

Lastly, hostmap improves availability by allowing multiple instances to run at the same time, but does not coordinate these instances. This trades strict consistency for simplicity, something that DBC Digital requested, but could be explored further for future work.

7.4 Limitations of Using Telemetry for Runtime Provenance

An important question to answer is whether existing technologies or frameworks could have been used to implement Hostmap, or at least serve as a foundational part of it.

As described in Section 2.4.3 and Section 2.4.4, both OpenTelemetry and Prometheus are telemetry systems. Their primary purpose is to provide insight into the health and behaviour of infrastructure by gathering and analysing telemetry data. Though they are well-suited in their domain, their design choices and the guarantees they provide are incompatible with hostmap's goals.

The core requirement is being able to determine which system image a given host was running at a given time and to maintain a complete history of system configuration state over time. If data is missing, this undermines the goal to audit deployments across time.

In the Section 2.4.3, describing OTel, we learn that the framework sends telemetry signals with a best-effort attempt. There are only guarantees between a single client and server hop, and it doesn't guarantee end-to-end delivery across multi-stage pipelines, which is something that hostmap would have needed. Because of this, telemetry signals may be lost without detection. This is not acceptable for hostmap, because data loss without detection is incompatible with runtime provenance. Missing activation data would make auditing difficult or impossible.

A more promising approach is Prometheus, described in Section 2.4.4. Since it uses a pull-based model, Prometheus does not rely on best-effort push delivery, which bypasses the problem OTel had.

Each scraping of data is a snapshot of one or multiple samples at a given time, rather than an authoritative record of the state over time.

In Prometheus, there are no lists of structured data, only single sample values. Its data model, as described in Section 2.4.4, does not provide an authoritative record of state of all state across time but instead records state at certain timestamps. Because data is sampled on a timer, a system could activate a new system configuration and later change again between scrapes without the activation being recorded.

As a result, both Prometheus and OTel are well-suited for monitoring, but not for seeing the exact state of configurations or activations over time to provide runtime provenance.

Thus I had to implement the functionality for saving the activation data and mappings to fulfil the requirements for runtime provenance and for this reason hostmap treats system activations as discrete events that must be recorded and persisted.

7.5 Future Work

A third interesting direction is seeing if a system similar to hostmap could be implemented for something like SLSA's build-time provenance. A hostmap-like system could consume SLSA provenance hashes and combine them with runtime state collected from hosts, providing a view into which artifacts are actually running across a server fleet.

There are several improvements to the system that could be valuable. One idea is that, if a host in a group is on a different commit than the rest, hostmap could trigger an alert or redeploy it. Another idea is to provide more advanced querying, to allow the web interface to "show all hosts running a system built from commit Y" or "list all commits that produced this store path". The data for implementing this is already in place.

Another area where hostmap could improve is log rotation on hosts and incremental scraping (fetching only new lines since the last scrape). This would reduce bandwidth and disk usage. On the server side, correlating the activations to mappings could be supplemented with caching to keep the UI fast as the database grows.

7.6 Conclusion

The goal of the project was to address the need for a reliable, secure and quick-to-update overview of NixOS hosts in a server fleet.

The result of this work is hostmap. It replaces the legacy system with a more robust and effective one. Through clear separation of responsibilities and a focus on small components with few responsibilities hostmap improves security, data freshness and reliability.

After having been tested and evaluated thoroughly and used in the day-to-day operations of the infrastructure team, it has been confirmed that hostmap achieves all of the requirements and provides real-world value.

Overall, this project demonstrates that it is possible to build an easy-to-deploy system for runtime provenance in NixOS environments and that by using careful system design, a highly-coupled legacy system can be replaced with a solution that is practical to use and secure.

Bibliography

- [1] NixOS Contributors. *How Nix Works*. URL: <https://nixos.org/guides/how-nix-works/> (visited on 12/18/2025).
- [2] NixOS Wiki Contributors. *Flakes*. URL: <https://wiki.nixos.org/wiki/Flakes> (visited on 12/18/2025).
- [3] David-Paul Dornseifer and Ben Liderman. *Establishing Verifiable Security: Reproducible Builds and AWS Nitro Enclaves*. URL: <https://aws.amazon.com/blogs/web3/establishing-verifiable-security-reproducible-builds-and-aws-nitro-enclaves> (visited on 12/19/2025).
- [4] Individual Contributors. *Cryptography - Use of OpenSSL*. URL: <https://cryptography.io/en/46.0.3/> (visited on 12/16/2025).
- [5] Morph contributors. *Morph: NixOS Deployment Tool*. URL: <https://github.com/DBCDDK/morph> (visited on 12/11/2025).
- [6] OpenTelemetry Authors. *OTLP Specification 1.9.0*. URL: <https://opentelemetry.io/docs/specs/otlp/> (visited on 12/15/2025).
- [7] Prometheus Authors. *Prometheus - Data model*. URL: https://prometheus.io/docs/concepts/data_model/ (visited on 12/16/2025).
- [8] The Open Source Security Foundation. *SLSA v1.2 — Principles*. URL: <https://slsa.dev/spec/v1.2/principles> (visited on 12/12/2025).
- [9] Red Hat. *What is GitOps?* URL: <https://www.redhat.com/en/topics/devops/what-is-gitops> (visited on 12/12/2025).
- [10] Agile Manifesto Authors. *Manifesto for Agile Software Development*. URL: <https://agilemanifesto.org/> (visited on 12/19/2025).
- [11] Wikipedia Authors. *Model–view–controller*. URL: <https://en.wikipedia.org/wiki/Model%E2%80%93view%E2%80%93controller> (visited on 12/16/2025).
- [12] Wikipedia contributors. *Timing attack*. URL: https://en.wikipedia.org/wiki/Timing_attack (visited on 12/07/2025).
- [13] PostgreSQL Global Development Group. *8.14. JSON Types*. URL: <https://www.postgresql.org/docs/18/datatype-json.html> (visited on 12/10/2025).
- [14] SQLx Maintainers. *Sqlx Documentation*. URL: <https://docs.rs/sqlx/0.8.6/sqlx/> (visited on 12/10/2025).
- [15] SQLx Maintainers. *Sqlx CLI*. URL: <https://crates.io/crates/sqlx-cli/0.8.6> (visited on 12/09/2025).

A SQL Query

This appendix shows the SQL query used by the back end to match each host's latest activation with its Git commit mappings.

```
1 WITH latest AS (  
2     SELECT  
3     DISTINCT ON (ac.hostname)  
4         ac.activation_id, ac.activated_at,  
5         ac.username, ac.store_path,  
6         ac.activation_type, ac.hostname  
7     FROM activation ac  
8     ORDER BY ac.hostname, ac.activated_at DESC  
9 )  
10 SELECT DISTINCT ON(l.hostname)  
11     l.activation_id, l.activated_at,  
12     l.username, l.store_path,  
13     l.activation_type, l.hostname,  
14     ngl.commit_hash, ngl.branch  
15 FROM latest l  
16 LEFT JOIN nix_git_link ngl ON ngl.store_path = l.  
17     store_path  
18 ORDER BY l.hostname, ngl.branch = 'master' desc, ngl.  
19     linked_at asc  
20 ;
```

B Feedback in Danish

The feedback has been lightly edited for clarity.

B.1 Sarah Brofeldt — Lead Infrastructure Developer

Sarah gave the following written feedback on the system:

Jeg synes, at det absolut vigtigste for mig er det forbedrede backend-design, altså at

1. det er HA/redundant (så jeg trygt kan miste en af hosts-p...)
2. det er dekoblet fra vores CI-servere (så de ikke længere er special cases, men blot kan nedlægges og genskabes)
3. det har en meget sikrere snitflade (simpel, isoleret HTTP-scrape frem for en, om end meget upriviligeret, bred SSH-adgang på tværs af hosts)

Men jeg har også brugt gui-forbedringerne rigtig meget, hvilket faktisk kom som en overraskelse f.eks. har jeg ifbm. opdatering af vores hosts til NixOS 25.11 brugt filtrer-featureen på "25.05" for at finde uopdaterede hosts (modulo dem der ikke har nyt hostmodul endnu)

Jeg har også været rigtig glad for især den forbedrede worst-case opdateringsfrekvens

B.2 Jonathan Zielinski — Infrastructure Developer

Jonathan gave the following written feedback on the system:

Vores tidligere hostoverview trængte til en modernisering

- Ingen af os skriver PHP på en god dag
- Takket være det nye søgefelt og den øgede opdateringsfrekvens, har det nye system nu erstattet det gamle i min daglige arbejdsgang.

B.3 Erik Skytthe — Infrastructure Developer

Erik gave the following written feedback on the system:

Jeg bruger nu hostmap-oversigten løbende under vores 14-dages opdateringer. Det er virkelig godt, at oversigten nu opdateres ret hurtigt, så man ikke skal vente omkring 10 minutter på at se resultatet. Jeg synes også dit indbyggede nix-diff i interfacet er ret sejt. smart, direkte links til commits i gitlab!

Technical
University of
Denmark

Anker Engelunds Vej 101
2800 Kgs. Lyngby
Tlf. 4525 1700

www.byg.dtu.dk